

Heuristic Bandwidth Reduction Algorithms for Weighted Graphs

- **Layout** of G : a bijection $\pi : V(G) \rightarrow \{0, 1, \dots, |V(G)| - 1\}$
- **Bandwidth** of G under layout π : $\beta_\pi(G) = \max_{\{u,v\} \in E(G)} |\pi(u) - \pi(v)|$
- Interested in finding **low-bandwidth layouts** because of applications in optimizing circuit layout, speeding up PDE solvers, etc.
- e.g., paths have **minimum bandwidth** $\beta(P_n) = 1$, stars have $\beta(S_n) = \lfloor n/2 \rfloor$, and complete graphs have $\beta(K_n) = n - 1$
- For a **weighted graph**: $\beta_\pi(G) = \max_{\{u,v\} \in E(G)} w(u, v) \cdot |\pi(u) - \pi(v)|$, where w is the weight function
- Most algorithms are for unweighted graphs only; these neglect, e.g., non-uniform **timing criticality** when designing circuits

Designing Heuristic Algos. for Weighted Bandwidth

- Usually, engineers/et al. simply use unweighted algorithms and accept this simplification, but we can do better ...
- **Reverse Cuthill–McKee** (RCM) and other heuristics *can* be trivially adapted to weighted graphs, but this is rather suboptimal
- My (very early-stages) **“Big Bear 1011”** algorithm:
 - ① Takes a linear forest with high-weight edges, then contracts all paths (computing the weights of the new edges is nontrivial)
 - ② Runs both weighted RCM and a custom RCM where all edges below a certain weight threshold are (temporarily) deleted
 - ③ Re-expands the super-nodes and returns the better layout
- Rmk.: Custom RCM wins on “heavy-backbone” graphs (e.g., **VLSI circuits** or **caterpillars**); weighted wins on fairly uniform graphs



Observe: Big Bear in Action!

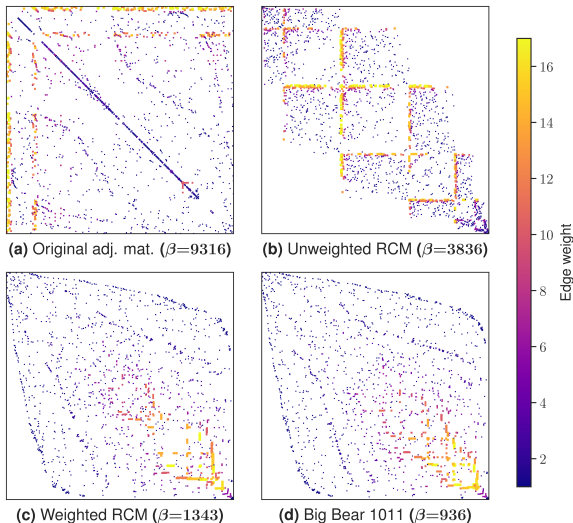


Figure: “Big Bear 1011” reduces β by >30% more than weighted RCM on a 561-cell circuit benchmark (s1196, ISCAS 1989). [Code: github.com/Luis-Varona/eccc-2026-weighted-bw.]